

Informatics ITS Graph Theory IUP

Group 1 Assignment 2

Members:

1. Khumaidy Syafiq El Maududy (5025251012)
2. Agile Octa Agrakha Handrian (5025251010)
3. Keven John Gondowardojo (5025251016)
4. Renato Kiran Arisandi (5025251015)

1. Prim's Algorithm Report

1.1 Overview & Theoretical Principles

Prim's algorithm is a greedy vertex-based procedure that computes a Minimum Spanning Tree (MST) $T = (V, E_T)$ in a connected, undirected, edge-weighted graph $G = (V, E)$, connecting all $|V|$ vertices using $|V|-1$ acyclic edges to minimize total weight $w(T) = \sum w(e)$.

Unlike Kruskal's algorithm, which sorts and evaluates edges globally, Prim's algorithm grows a single tree locally, one vertex at a time, always expanding across the cheapest available boundary.

Cut Property Theorem (shared with Kruskal): For any cut $(S, V \setminus S)$, the lightest cut-crossing edge belongs to an MST. Prim's algorithm applies this theorem directly and repeatedly: at every step, the current tree U and the remaining vertices $V \setminus U$ define a cut, and the lightest edge crossing that cut is guaranteed safe to add. If omitted, adding it later would only be possible via a heavier or equal crossing edge, which cannot improve the tree — proving optimality by substitution.

Cycle Property (implication, not enforcement): Because Prim's algorithm only ever selects edges that cross the current cut (one endpoint inside the tree, one outside), a cycle can never be formed by construction. This is a structural difference from Kruskal's algorithm, which must actively test and reject edges whose endpoints already share a component. Prim's algorithm never needs this test — cycles are impossible by definition of the frontier.

1.2 Algorithm Steps & Frontier (Cut Set) Maintenance

Kruskal's algorithm keeps track of components with a Disjoint Set Union (DSU). Prim's algorithm doesn't need that — instead we just keep a list of "frontier" edges, meaning every edge that has one end inside the tree and one end outside.

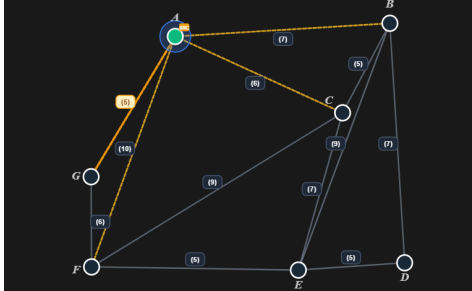
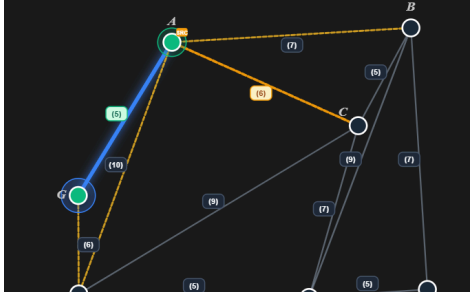
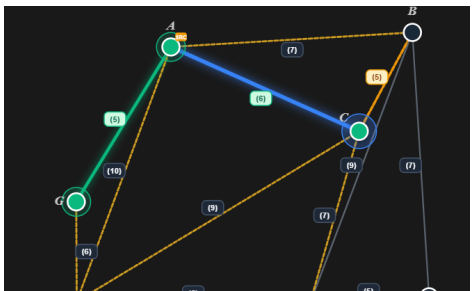
Here's how we did it step by step:

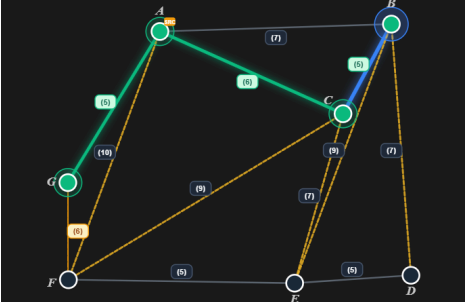
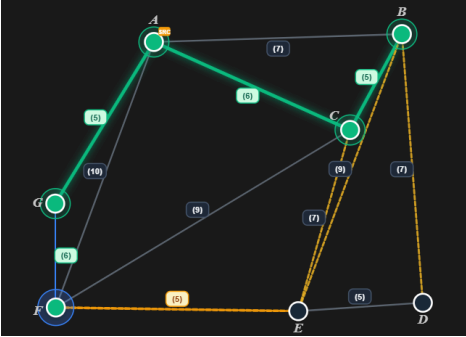
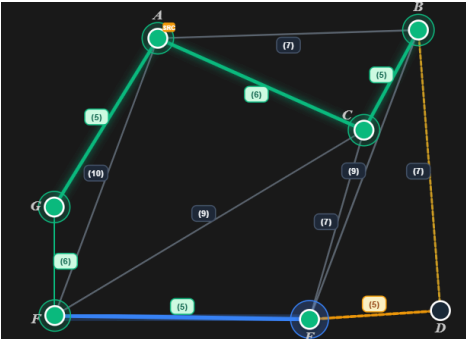
1. Pick any starting vertex and put it in the tree (call this set U).
2. Look at all edges where one vertex is already in U and the other one isn't. This list of edges is the frontier.
3. From that frontier, pick the edge with the smallest weight and add it to the tree. Whichever new vertex that edge connects to now also joins U .
4. Update the frontier: any edge that now has both ends inside U gets dropped (it's not a "way out" anymore), and any new edges from the vertex we just added get added to the list.
5. Keep repeating steps 2–4 until every vertex is in U . Once we've added $|V| - 1$ edges, we're done and the tree is the MST.

The main idea is: we never look at the whole edge list like Kruskal does. We only ever compare edges that are currently on the "border" of the tree, and we always take the cheapest one available at that moment.

1.3 Step-by-Step Execution Trace

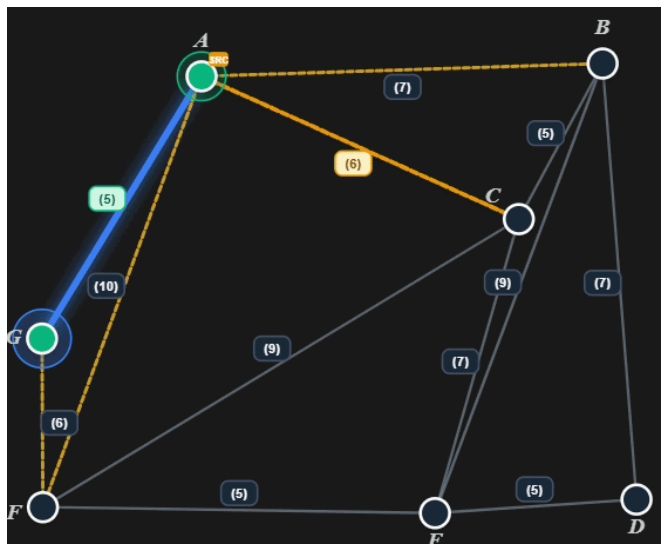
Step	Edge (u,v)	Weight	Himpunan Pohon (Tree Set U)	Status / Keterangan	Graph
------	------------	--------	-----------------------------	---------------------	-------

0	-	-	{A}	Initialization, start from A	
1	(A,G)	5	{A,G}	ACCEPTED (minimum edge crossing cut {A} rest)	
2	(A,C)	6	{A,C,G}	ACCEPTED (minimum edge crossing cut {A,G} rest; beat A-B:7, A-F:10, G-F:6)	

3	(C,B)	5	{A,B,C,G}	ACCEPTED (minimum edge crossing cut {A,C,G} rest; beat A-B:7, C-E:7, C-F:9, G-F:6)	
4	(G,F)	6	{A,B,C,F,G}	ACCEPTED (minimum edge crossing cut {A,B,C,G} rest; beat B-D:7, B-E:9, C-E:7, C-F:9)	
5	(F,E)	5	{A,B,C,E,F,G}	ACCEPTED (minimum edge crossing cut {A,B,C,F,G} rest; beat B-D:7, B-E:9, C-E:7)	

6	(E,D)	5	{A,B,C,D,E, F,G}	ACCEPTED MST complete cost = 32	
---	-------	---	---------------------	---	--

Result Summary: MST Edges: (A-G: 5), (A-C: 6), (C-B: 5), (G-F: 6), (F-E: 5), (E-D: 5) | **Total Cost: 32.0**



2. Kruskal's Algorithm Report

2.1 Overview & Theoretical Principles

Kruskal's algorithm is a greedy edge-based procedure that computes a Minimum Spanning Tree (MST) $T = (V, E_T)$ in a connected, undirected, edge-weighted graph G

$= (V, E)$, connecting all $|V|$ vertices using $|V|-1$ acyclic edges to minimize total weight $w(T) = \sum w(e)$.

Cut Property Theorem: For any cut $(S, V \setminus S)$, the lightest cut-crossing edge belongs to an MST. If omitted, adding it creates a cycle with a heavier/equal crossing edge e' ($w(e') \geq w(e)$), proving optimality by substitution.

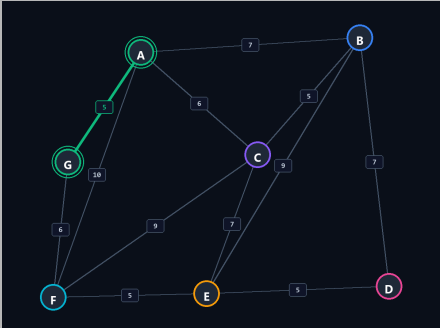
Cycle Property Theorem: The strictly heaviest edge $e_{\max}$ in any cycle C cannot belong to any MST. Kruskal enforces this by rejecting candidate edges whose endpoints share a component.

2.2 Algorithm Steps & Disjoint Set Union (DSU)

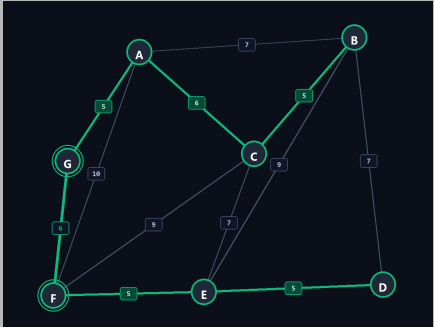
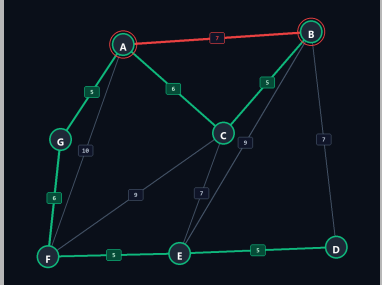
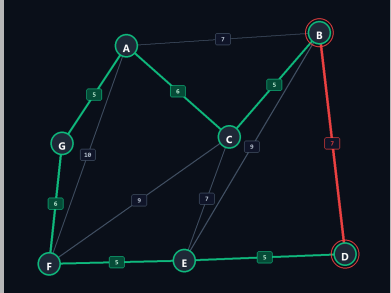
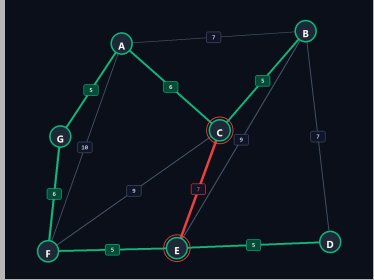
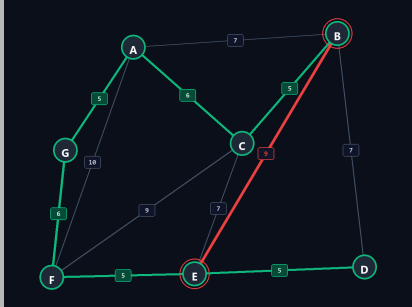
- Init:** $\text{MakeSet}(v)$ for all $v \in V$. (Start by putting every node/vertex in its own separate group.)
- Sort:** Order edges by weight $w(e_1) \leq \dots \leq w(e_{|E|})$. (Sort all connections/edges from the lightest/cheapest weight to the heaviest/most expensive.)
- Greedy Select:** For $e = (u, v)$, if $\text{Find}(u) \neq \text{Find}(v)$, accept e and $\text{Union}(u, v)$; else discard (cycle prevention). (Go through each edge in sorted order. if it connects two different groups without creating a closed loop/cycle, pick it and merge the groups; otherwise, skip it.)
- Halt:** Stop at $|V|-1$ edges. (Stop as soon as you have picked enough edges to connect all nodes into one tree.)

2.3 Step-by-Step Execution Trace (MIT Figure 6.11)

Reference graph: 7 vertices (A–G), 12 candidate edges.

Step	Edge (<i>u,v</i>)	Bobot	Himpunan Komponen (Disjoint Sets)	Status / Keterangan	Graf
0	– –	– –	{A},{B},{C},{D},{E},{F},{G}	Inisialisasi 7 komponen terpisah	–
1	(A,G)	5.0	{A,G},{B},{C},{D},{E},{F}	ACCEPTED (Gabung {A,G})	

2	(B,C)	5.0	{A,G},{B,C},{D},{E},{F}	ACCEPTED (Gabung {B,C})	
3	(E,D)	5.0	{A,G},{B,C},{D,E},{F}	ACCEPTED (Gabung {D,E})	
4	(F,E)	5.0	{A,G},{B,C},{D,E,F}	ACCEPTED (Gabung {D,E,F})	
5	(A,C)	6.0	{A,B,C,G},{D,E,F}	ACCEPTED (Gabung {A,B,C,G})	

6	(G,F)	6.0	{A,B,C,D,E,F,G}	ACCEPTED, MST complete cost = 32	
7	(A,B)	7.0	{A,B,C,D,E,F,G}	<i>REJECTED (Cycle: A-C-B-A)</i>	
8	(B,D)	7.0	{A,B,C,D,E,F,G}	<i>REJECTED (Cycle)</i>	
9	(C,E)	7.0	{A,B,C,D,E,F,G}	<i>REJECTED (Cycle)</i>	
10	(B,E)	9.0	{A,B,C,D,E,F,G}	<i>REJECTED (Cycle)</i>	

11	(C,F)	9.0	{A,B,C,D,E,F,G}	REJECTED (Cycle)	
12	(A,F)	10.0	{A,B,C,D,E,F,G}	REJECTED (Cycle)	

Result Summary: MST Edges: (A-G: 5), (B-C: 5), (E-D: 5), (F-E: 5), (A-C: 6), (G-F: 6) | Total Cost: 32.0

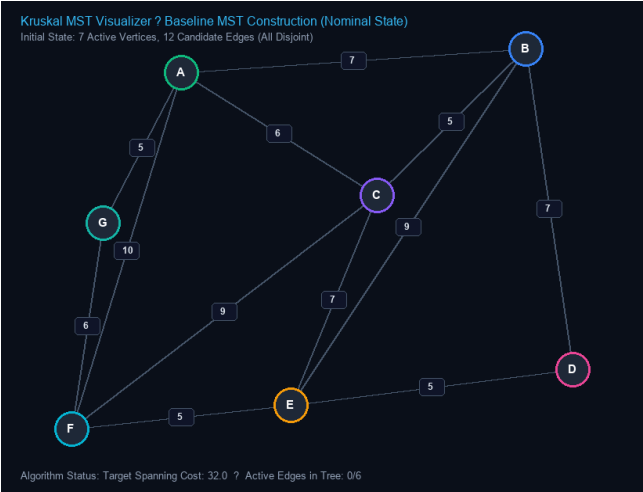


Figure 2.1 ? Step-by-step Kruskal MST construction on MIT Figure 6.11 reference network (Total Cost: 32.0).

3. Boruvska’s Algorithm Report

3.1 Overview & Theoretical Principles

Boruvka's algorithm is a greedy algorithm used to compute a Minimum Spanning Tree (MST) of a connected, undirected, weighted graph $G = (V, E)$. An MST connects all $|V|$ vertices using exactly $|V| - 1$ edges without forming any cycle while minimizing the total sum of edge weights.

The main idea of Boruvka's algorithm is to initially treat every vertex as an independent component. During each iteration, every component selects its cheapest outgoing edge, which is the minimum-weight edge connecting that component to another component. These selected edges are then added to the MST when they connect different components.

This process gradually merges the components until all vertices belong to a single connected component. Since each selected edge connects two different components, the resulting structure remains acyclic.

Cut Property: For any component S of the current forest, the minimum-weight edge connecting S to another component is a safe edge that can be included in an MST. Boruvka's algorithm applies this property simultaneously to all current components in every iteration.

Unlike Prim's algorithm, which grows a single tree from a starting vertex, and Kruskal's algorithm, which processes edges globally in increasing order, Boruvka's algorithm processes multiple components simultaneously by selecting the cheapest outgoing edge from each component.

3.2 Algorithm Steps & Disjoint Set Union (DSU)

Algorithmic Procedure:

1. **Initialization:** Create a separate component for every vertex in V .
2. **Find Cheapest Edges:** For every current component, examine its outgoing edges and select the edge with the minimum weight.
3. **Edge Selection:** Add each selected edge to the MST if its endpoints belong to different components.
4. **Component Merging:** Merge the components connected by the selected edges using the Union operation.
5. **Repeat:** Continue finding and selecting the cheapest outgoing edge for every component until all vertices belong to one component.

6. **Termination:** The selected edges form the Minimum Spanning Tree, containing $|V| - 1$ edges.

Disjoint Set Union (DSU):

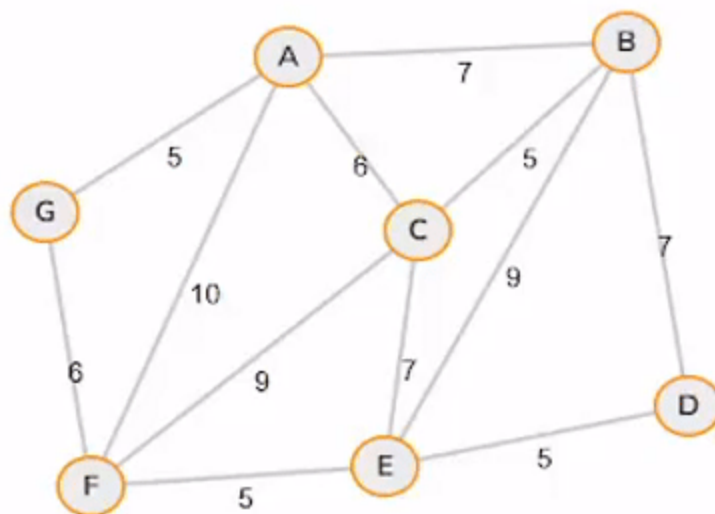
The implementation uses a Disjoint Set Union (DSU) structure to maintain the current components of the graph.

- **Find:** Determines the representative (root) of the component containing a vertex. Path compression is used to make repeated searches more efficient.
- **Union:** Merges two different components. If both vertices already have the same root, the edge is not added because it would create a cycle.

The DSU structure allows the algorithm to efficiently determine whether a selected edge connects two different components before adding it to the MST.

3.3 Step-by-Step Execution Trace

The following trace shows the execution of Boruvka's algorithm on the given graph. In each round, every component selects its cheapest outgoing edge. The selected edges are then used to merge different components. This process continues until all vertices are connected into a single component.



Step	Components	Cheapest Edge(s)	Components After Union	Graph
0	{A}, {B}, {C}, {D}, {E}, {F}, {G}	-	{A}, {B}, {C}, {D}, {E}, {F}, {G}	
1	{A}, {B}, {C}, {D}, {E}, {F}, {G}	(A-G)=5 (B-C)=5 (D-E)=5 (E-F)=5	{A,G}, {B,C}, {D,E,F}	
2	{A,G}, {B,C}, {D,E,F}	(A-C)=6 (G-F)=6	{A,B,C,D,E,F,G}	

Result Summary: The algorithm successfully connects all 7 vertices using 6 edges. The resulting Minimum Spanning Tree consists of (A-G: 5), (B-C: 5), (D-E: 5), (E-F: 5), (A-C: 6), and (G-F: 6), with a total MST cost of **32**.

4. Edge Case Simulation

A. Prim

Scenario 1: Critical Link Failure (Edge (G, F) Severed)

Failure Impact

Trunk edge (G, F) ($w=6.0$) is a critical artery connecting the western perimeter. Severing (G, F) removes the tree edge and partitions the vertices into two disconnected components: $S=\{A, B, C, G\}$ and $V \setminus S=\{D, E, F\}$. This leaves 5 unaffected baseline edges in the spanning forest.

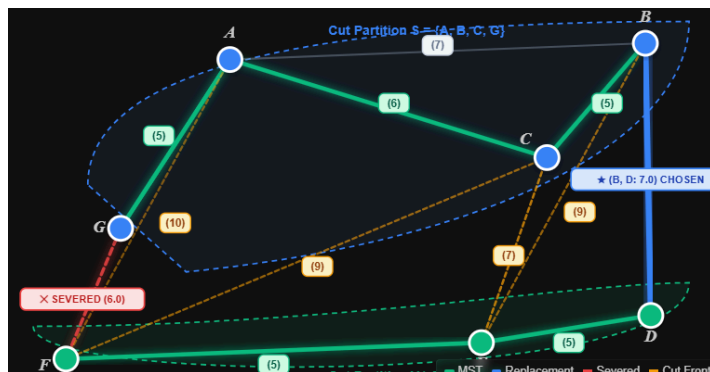
Prim Reconfiguration & Edge Selection

Rather than re-evaluating the entire graph, Prim's algorithmic framework naturally handles link failure via **localized cut re-scanning**. The cut across S and $V \setminus S$ is reopened with the following frontier candidate edges: $\delta(S) = \{(B, D) : 7.0, (C, E) : 7.0, (B, E) : 9.0, (C, F) : 9.0, (A, F) : 10.0\}$

- **Cheapest Frontier Edge:** Both (B, D) and (C, E) possess the minimum crossing weight of 7.0 .
- **Selection:** Per the Cut Property, Prim selects $(B, D : 7.0)$ to heal the partition.
- **Dynamic Advantage:** Only the local heap frontier crossing the ruptured cut is updated; no global sorting of unaffected edges is required.

Adapted Metrics

- **Selected Edges:**
 $(A-G : 5.0), (A-C : 6.0), (B-C : 5.0), (B-D : 7.0), (E-D : 5.0), (F-E : 5.0)$
- **Adapted Cost:** 33.0 ($\Delta = +1.0$ / $+3.125\%$ failure penalty)
- **Structural Preservation:** Retains **83.3%** of baseline edges (5 of 6 edges).
- **Average Edge Weight:** $33.0/6 = 5.50$ ($+3.19\%$)



Scenario 2: Hub Node Failure (Node C Destroyed)

Failure Impact

Central Hub C possesses degree 4. Destroying Node C eliminates all 4 incident edges: $\{(A, C) : 6.0, (B, C) : 5.0, (C, E) : 7.0, (C, F) : 9.0\}$. This leaves 6 active vertices ($V' = \{A, B, D, E, F, G\}$), necessitating $|V'| - 1 = 5$ edges to span the remaining subnetwork. Node B is completely disconnected from the lower perimeter.

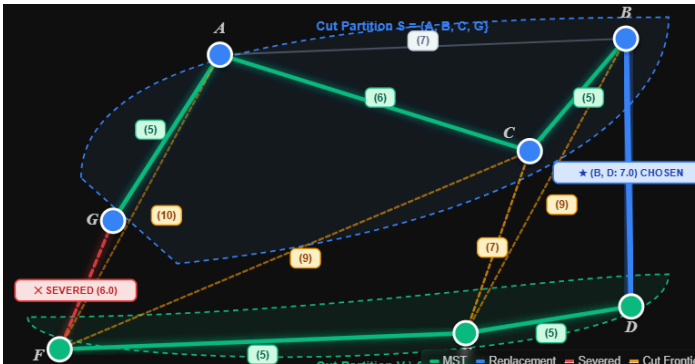
Prim Reconfiguration

Starting from root **A**, Prim executes on the remaining topology:

1. **Perimeter Expansion:** Greedily commits edges **(A-G:5.0)**, **(G-F:6.0)**, **(F-E:5.0)**, and **(E-D:5.0)**, forming a robust trunk spanning {A, D, E, F, G}.
2. **Reconnection of Node B:** The frontier crossing to isolated Node **B** consists of: $\delta(\{B\})=\{(A,B):7.0, (B,D):7.0, (B,E):9.0\}$ Prim selects the direct northern bypass **(A,B:7.0)** (or symmetric **(B,D:7.0)**), restoring complete connectivity.

Adapted Metrics

- **Selected Edges:** **(A-G:5.0)**, **(G-F:6.0)**, **(F-E:5.0)**, **(E-D:5.0)**, **(A-B:7.0)**
- **Adapted Cost:** **28.0** ($\Delta=-4.0$ / -12.5% reduction due to fewer required nodes)
- **Structural Preservation:** Retains **4** baseline edges (**80.0%** of the **5** required edges).
- **Efficiency Loss:** Average edge weight increases from **5.33** to **5.60** (**+5.07%**), reflecting the structural penalty of bypassing the central hub.



- Quantitative Evaluation & Comparative Matrix

Evaluation Metric	Baseline MST	Scenario 1: Edge Failure	Scenario 2: Node Failure
Failure Event Description	Nominal State (No Failures)	Edge (G, F) severed	Hub Node C destroyed
Active Vertices (V')	7 (A, B, C, D, E, F, G)	7 (A, B, C, D, E, F, G)	6 (A, B, D, E, F, G)
Required Spanning Edges (V' - 1)	6	6	5

Selected MST Edges	(A-G, A-C, C-B, G-F, F-E, E-D)	(A-G, A-C, C-B, F-E, E-D, B-D)	(A-G, G-F, F-E, E-D, A-B)
Total Tree Weight $w(T)$	32.0	33.0 (Delta = +1.0)	28.0 (Delta = -4.0)
Average Edge Weight	5.33	5.50 (+3.19%)	5.60 (+5.07%)
Network Topological State	Connected MST	Connected MST	Connected MST

-

B.

- **Scenario 1: Critical Link Failure (Edge (G, F) Severed)**
- **Failure Impact:** Severing trunk (G, F) [w=6.0] partitions vertices into $S=\{A, B, C, G\}$ and $V \setminus S=\{D, E, F\}$. Retains 5 unaffected baseline edges.
- **Edge Selection:** Kruskal scans cut-crossing edges (B-D: 7.0, C-E: 7.0, B-E: 9.0, **Kruskal**C-F: 9.0, A-F: 10.0) and selects (B, D: 7.0) per the Cut Property.

Adapted Cost: 33.0 (Delta: +1.0 / +3.125%). Retains 83.3% of baseline structure.

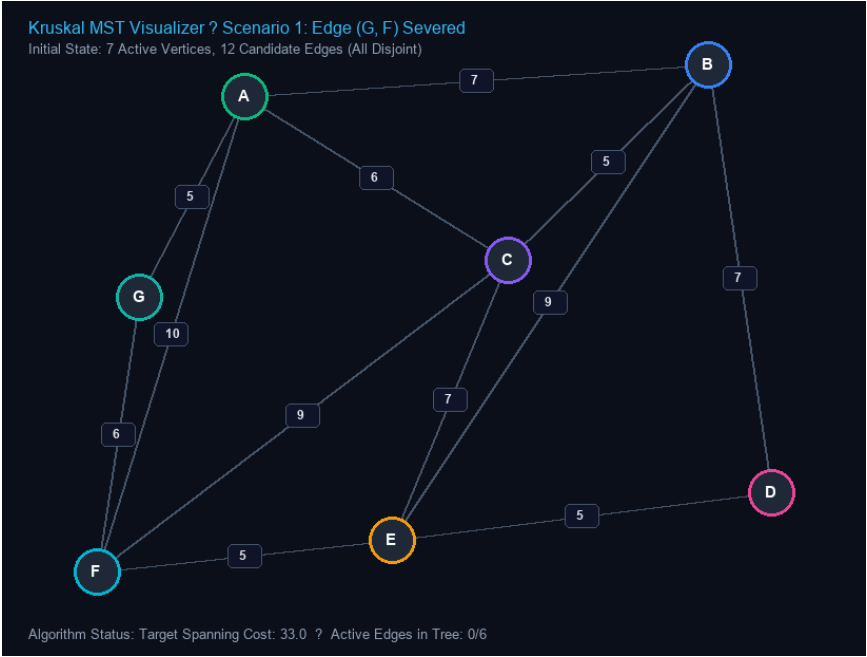


Figure 4.1 ? Dynamic tree re-routing after critical edge (G, F) failure, selecting replacement link (B, D) at weight 7.0 (Total Cost: 33.0).

– **Scenario 2: Hub Node Failure (Node C Destroyed)**

- **Failure Impact:** Destroying hub C removes incident edges (A-C: 6, B-C: 5, C-E: 7, C-F: 9), leaving 6 active vertices ($|V'| - 1 = 5$ required edges).
- **Kruskal Reconfiguration:** Accepts edges (A-G: 5), (E-D: 5), (F-E: 5), and (G-F: 6) to span $\{A, D, E, F, G\}$, then reconnects B via (A, B: 7.0).

Adapted Cost: 28.0 (Delta: -4.0 / -12.5%; retains 4 baseline edges).

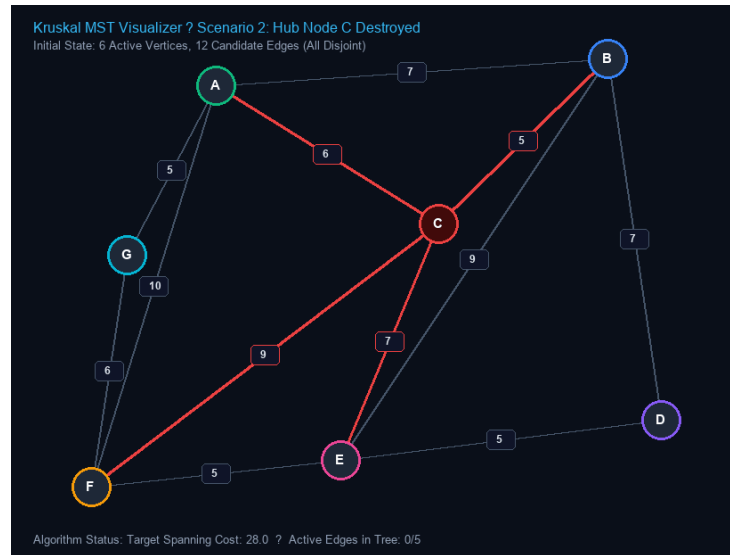


Figure 4.2 ? Subnetwork reconfiguration after Hub Node C failure, reconnecting Node B via edge (A, B: 7.0) across 6 active vertices (Total Cost: 28.0).

– **Quantitative Evaluation & Comparative Matrix**

Evaluation Metric	Baseline MST	Scenario 1: Edge Failure	Scenario 2: Node Failure
Failure Event Description	Nominal State (No Failures)	Edge (G, F) severed	Hub Node C destroyed
Active Vertices (V')	7 (A, B, C, D, E, F, G)	7 (A, B, C, D, E, F, G)	6 (A, B, D, E, F, G)
Required Spanning Edges ($V' - 1$)	6	6	5
Selected MST Edges	(A-G, B-C, E-D, F-E, A-C, G-F)	(A-G, B-C, E-D, F-E, A-C, B-D)	(A-G, E-D, F-E, G-F, A-B)
Total Tree Weight $w(T)$	32.0	33.0 (Delta = +1.0)	28.0 (Delta = -4.0)
Average Edge Weight	5.33	5.50 (+3.19%)	5.60 (+5.07%)

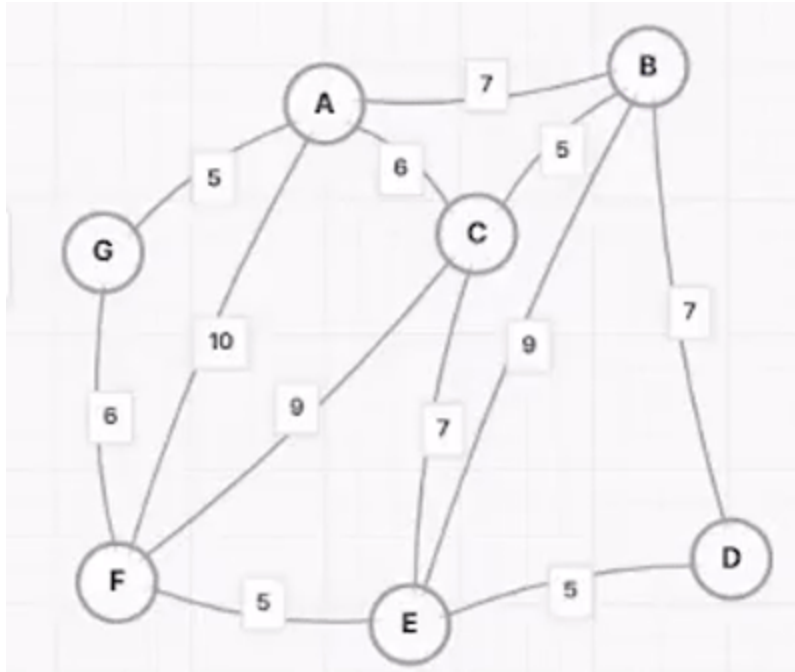
Network Topological State	Connected MST	Connected MST	Connected MST
- Edge Failure Penalty: Calculated as substitute edge minus severed edge weight ($7.0 - 6.0 = +1.0$). - Node C Destruction: Reduces total tree weight from 32.0 to 28.0 due to requiring fewer edges.			
Efficiency Loss: Average edge weight increases from 5.33 to 5.60 (+5.07%), reflecting reduced topological efficiency when bypassing the hub.			

C. Boruvka

3.4 Failure Case Simulation

- **Scenario 1:** Critical Link Failure (Edge (F, E) Severed)
- **Failure Impact:** Severing edge (F, E) [$w=5.0$] removes one of the baseline MST edges and leaves 5 unaffected baseline edges.
- **Edge Selection:** Borůvka recalculates the cheapest outgoing edge for each component using the remaining graph. Edge (C, E) [$w=7.0$] is selected as the replacement edge to reconnect the components.

Adapted Cost: 34.0 (Delta: +2.0 / +6.25%). Retains 83.3% of baseline structure.



- - **Scenario 2:** Junction Node Failure (Node C Destroyed)
- **Failure Impact:** Node C [and all edges connected to C] is removed from the graph, leaving 6 active vertices and 4 unaffected baseline MST edges.
- **Edge Selection:** Borůvka recalculates the cheapest outgoing edge for each component using the remaining graph. Edge (A, B) [w=7.0] is selected as the replacement edge to reconnect the components.

Adapted Cost: 28.0 (Delta: -4.0 / -12.5%). Retains 66.7% of baseline structure.

